

Strategy Pattern debugging guide

A clean live-demo guide for proving Context -> Strategy -> ConcreteStrategy in the actual FastAPI backend.

Strategy role	Project implementation	What to prove
Context	SeparationEngine app/ml/separation_engine.py	Runs separation through self.algorithm instead of hardcoding NeoSSNet.
Strategy interface	SeparationAlgorithm Protocol app/ml/separation_algorithm.py	Defines the common separate(...) contract for all algorithms.
ConcreteStrategy	NeoSSNetStrategy app/ml/neossnet_strategy.py	The real runtime algorithm used for NeoSSNet separation.
Setup point	SeparationService app/services/separation_service.py	Creates SeparationEngine and injects the selected algorithm object.

Main proof sentence

The Strategy Pattern is proven when SeparationEngine calls self.algorithm.separate(...). The variable self.algorithm is typed as the SeparationAlgorithm interface, but at runtime the object is NeoSSNetStrategy.

Why not breakpoint inside SeparationAlgorithm?

SeparationAlgorithm is a Protocol interface. It defines the method contract but does not run real code. During debugging, VS Code will stop in the Context and the ConcreteStrategy, not inside the interface body. This is normal and correct.

If the browser waits during the demo

When VS Code stops at a backend breakpoint, the browser request is paused. The web page may keep loading until you press Continue. This proves the request reached the backend code; it is not a frontend problem.

Say: "The browser is waiting because FastAPI is paused at the breakpoint. After I press Continue, the request finishes and the page updates."

Step-by-step breakpoint demo

Use only these Strategy breakpoints. Disable unrelated breakpoints before starting.

Step	Breakpoint / action	What to press or inspect	What it proves
1	Set breakpoint at app/services/separation_service.py:138 engine = self.engine_class(algorithm=algorithm, device_name="cpu")	Click Run Separation. Inspect algorithm after line 137.	The selected algorithm object is prepared before the engine runs. It should be NeoSSNetStrategy.
2	At line 138, press Step Over.	Inspect engine.algorithm.	The Context, SeparationEngine, now holds the algorithm object.
3	Continue to app/ml/separation_engine.py:29 return self.algorithm.separate()	Inspect self.algorithm, then press Step Into.	The Context calls the Strategy through the interface-style variable self.algorithm.
4	VS Code enters app/ml/neossnet_strategy.py:23 if model_config_path is None:	Point to the file name and class name NeoSSNetStrategy.	The real runtime object is the ConcreteStrategy.

Expected debugger flow

```
SeparationService prepares algorithm -> SeparationEngine Context -> self.algorithm.separate(...) ->
NeoSSNetStrategy ConcreteStrategy
```

What to say during the demo

- Now I will demonstrate the Strategy Pattern using the separation engine.
- First, SeparationService prepares the selected algorithm object. I can inspect the algorithm variable here, and the runtime object is NeoSSNetStrategy.
- Next, SeparationService creates SeparationEngine and passes the algorithm into it. This means the engine does not create or hardcode NeoSSNet by itself.
- Now execution stops inside SeparationEngine. SeparationEngine is the Context.
- At this line, the Context calls self.algorithm.separate(...). This is the Strategy call.
- When I Step Into, execution enters NeoSSNetStrategy. That proves the runtime ConcreteStrategy is NeoSSNetStrategy.
- The benefit is maintainability. Later, another algorithm can implement the same SeparationAlgorithm contract, and the engine workflow does not need to be rewritten.

Quick lecturer explanation

Use this if you need a shorter explanation without stepping too deeply into ML code.

Short version

The Strategy Pattern separates the algorithm from the engine. SeparationEngine is the Context. It depends on the SeparationAlgorithm contract and calls `self.algorithm.separate(...)`. The current real algorithm object is `NeoSSNetStrategy`. Because of this, we can add another algorithm later without changing the engine workflow.

What to inspect in VS Code

Where	Variable/object	What lecturer should see
separation_service.py:138	algorithm	Runtime object is <code>NeoSSNetStrategy</code> .
separation_service.py:138 after Step Over	engine.algorithm	SeparationEngine stores the selected strategy object.
separation_engine.py:29	self.algorithm	Context calls <code>separate()</code> through the algorithm reference.
neossnet_strategy.py:23	self / file name	Execution is now inside <code>NeoSSNetStrategy</code> .

What not to show

- Do not step deeply into `run_neossnet_inference` unless the lecturer asks about ML internals.
- Do not call `SeparationService` the Strategy Context. In the final mapping, the Context is `SeparationEngine`.
- Do not say `SeparationAlgorithm` executes real code. It is the interface/contract.
- Do not wait too long if local ML inference is slow. Stop after proving the design boundary.

If inference fails locally

Say: "Even if local inference fails because of model files or environment setup, the Strategy Pattern is already proven because the request reached `SeparationEngine` and Step Into entered `NeoSSNetStrategy`."